

Research CLAUDE

A deeper map of image-to-garment, image-to-pattern, and nesting — and what Maya's backend Op Room actually needs to replace CLO3D.

Mana Siyo · Maya backend research dossier · v0.1 · May 19, 2026

Abstract

The earlier dossier (*Image to Garment Pattern Generation for a Local Backend System*) settled the upstream half of the question: the field has converged on **structured pattern representations** rather than raw pixel patterns, and the production-realistic stack is GarmentCode as the canonical language, ChatGarment or AIpparel as the parser, NGL-Prompter as the prompting template, and GarmentDiffusion as a fast helper. That answers what *image to garment* should produce. It does not answer what happens next.

This dossier picks up where that one stopped. Maya's backend already does *image to garment*: a dream-garment image is dissected into wearable items, each rendered alone on black. The Op Room then asks two harder questions. First, **image to pattern** — how do we turn a rendered piece into a real, sewable pattern with panels, grainlines, seam allowance, and notches? Second, **nesting** — once the graded pattern exists, how do we lay it out on fabric or on a laser bed with the least possible waste? The Operating Room is meant to replace CLO3D for both. This document lays out the landscape, the math, the open-source and commercial options, and a concrete implementation plan staged in three phases.

Part 1 · Where the field stands today

Production garment-AI work is converging on a clean three-layer pipeline: a semantic parser that ingests an image or sketch and emits structured JSON; a deterministic compiler that turns that JSON into 2D pattern geometry; and a manufacturing post-processor that injects grainlines, seam allowance, notches, ease, and closure templates before anything is cut. Academic benchmarks measure the first two layers. They do not standardize the third, which is why research-state SOTA is not manufacturing-complete.

The shorthand that came out of the prior dossier still holds: **edge predictors** (SewFormer, Panelformer, NeuralTailor) are research baselines. **Language-mediated systems** (ChatGarment, AIpparel, Design2GarmentCode, NGL-Prompter) are the production direction. GarmentCode is the canonical representation everyone in the second camp targets. The 2026 commercial landscape — Style3D, Synthesis by Six Atomic, StitchLift, plus the established Optitex / Lectra / Browzwear / CLO3D — essentially confirms this shape, with each vendor making different choices about what they expose, what they lock, and what they price.

How Maya sits relative to that map

Maya's frontend operates above the parser layer: it captures the consultation, extracts fashion refs, and renders three-view full-body visions via gpt-image. Maya's backend Op Room operates in the compiler and post-processor layers: a designer picks a saved client, lands on the one-pager, then swipes down to a 30/70 Operating Room. The left pane (Item Window) shows the active garment piece. The right pane (Pattern Window) has two horizontal pages — Pattern and Nest — with *Dissect* and *Nest* buttons. As of backend v0.10, Dissect still emits a decorative pixel image of a pattern via gpt-image-2; Nest is a placeholder that mirrors that image. Neither produces real geometry yet. Closing that gap is what this dossier is about.

Part 2 • Image to garment (what Maya already does)

Image to garment is the friendliest step in the pipeline because it can be solved with off-the-shelf vision language models and image-edit endpoints. Maya's current implementation is a tight two-stage pass.

Stage A — dissection. GPT-4.1 is asked, with a strict JSON-only system prompt, to identify the separate *wearable items* in a three-view atelier sheet. Items are returned with name, category (outerwear / top / bottom / dress / accessory / base), fabric, construction note, an approximate panel count, and a skip flag for base layers. Critical guardrails in the prompt: *items, not panels*; the base layer (plain black tank top and trousers Maya's avatar arrives in) is detected and skipped; complexity is tiered standard vs. avant-garde.

Stage B — isolated render. For each non-skip piece, the backend calls `/v1/images/edits` with the dream-garment as the base image and a prompt that asks for the piece alone, no model or mannequin, on a pure black background, in a three-view horizontal canvas (side / front / back). The renders land back on the piece objects (`piece._image`) so the swiper can flip between them without re-rendering.

What works. Designers can scrub through the pieces and see each garment item in isolation. The pure-black backdrop reads as an atelier reference. Dissection is conservative about hardware and fabric, which is what we want — downstream stages need to trust the call or override it deliberately.

What is missing. The output of this stage is pictorial, not structural. We have a picture of each garment item. We have no panel topology, no measurements, no notion of which seams pair with which, no closures listed by template. The next stage cannot be done by asking gpt-image-2 to *draw* a pattern. We have to step out of the image and into a structured representation.

Part 3 • Image to pattern (the hardest gap)

This is the layer where the field actually disagrees with itself. Three viable paths exist for getting from a rendered garment piece to something cuttable. They differ in fidelity, time cost, and how much we depend on third parties.

Path A • Raster pattern + post-trace

What we do today: ask gpt-image-2 to render a flat-pattern visualisation that *looks like* a CLO3D pattern window — white paper background, dark-grey panel outlines, dashed grainlines, small triangular notches. Then run a vectorizer (Vectosolve, potrace, ImageTracer.js, commercial PNG-to-SVG tools) to convert the pixel pattern into SVG paths.

Pros. Already wired. No model training. Fast to iterate visually.

Cons. The output is a single SVG containing a soup of paths. There is no labelled *front bodice* polygon, no *back bodice*, no sleeve. Vectorizers have no concept of seam pairing. A traced grainline is just a dashed stroke, not metadata. Any seam allowance is whatever the LLM happened to draw. The image is

a drawing of a pattern, not a pattern; trusting it for production would burn fabric. Use this only as a designer preview, never as the file you cut from.

Path B · Structured prompt to GarmentCode JSON

Recommended by the prior dossier. Replace the gpt-image-2 call with a gpt-4.1 call that returns a strict **GarmentCode-style JSON configuration** for the active piece: panel components, body-anchor parameters, stitch topology, closure type, grain direction. Then compile that JSON deterministically into SVG panels via PyGarment, the official GarmentCode Python library.

Why this is the right shape: it splits the problem. The LLM only has to be *semantically* correct (an A-line skirt has two panels, a waistband, and a back zipper closure); it does not have to draw correct geometry. The compiler is deterministic and tested. Grainline, seam allowance, and notches are injected by the compiler, not requested from the model. This is the same lesson the Design2GarmentCode and NGL-Prompter papers landed on.

Pros. Output is real geometry with metadata. Editable. Compileable to DXF ASTM D6673 for industry interchange. Aligns Maya with where the field is going.

Cons. Requires PyGarment running either server-side (Vercel Python function) or in-browser via Pyodide (heavier first load). Coverage of garment types is limited to what GarmentCode's component library supports — solid for common silhouettes, weaker on avant-garde construction (which is exactly Maya's sweet spot).

Path C · Local foundation model (ChatGarment / AIpparel)

The next-generation version of Path B: instead of asking GPT-4.1 for JSON, run a self-hosted ChatGarment or AIpparel checkpoint locally. ChatGarment is a fine-tuned LLaVA-1.5 7B that outputs GarmentCode JSON directly from a real garment photograph. AIpparel is a multimodal foundation model trained on 120k+ garments under MIT. Both are released; both need a real GPU host.

Pros. Better real-image robustness than prompting a general VLM. No API spend. Editable JSON output. Future-proof — both models are aligned with GarmentCodeData and improving.

Cons. Requires a GPU box (24–48GB class for AIpparel, 40–80GB class for ChatGarment per the paper). Adds an inference latency budget (50s/sample for ChatGarment on an A100, dominated by the stitching stage). Not a today's-session build.

Recommendation for Maya

Stage the migration. **Phase A** (this session and the next): keep Path A as the designer's visual preview. Do not throw it away — it is a useful aesthetic anchor in the Pattern Window. **Phase B:** stand up Path B in parallel. Add a second tab in the Pattern Window — *JSON* next to *Pattern* and *Nest* — that shows the GarmentCode JSON returned by the dissect call and previews the PyGarment-compiled SVG. Once Phase B is reliable, demote Path A to a preview-only thumbnail. **Phase C:** when GPU infrastructure

exists, swap the JSON-producing model from GPT-4.1 to a local ChatGarment or AIpparel checkpoint.

Industry-format interchange — DXF ASTM D6673 / AAMA

Whichever path produces the geometry, the file we export has to be readable by real cutting hardware and by Optitex / Lectra / Gerber CAD. The standard is **ASTM D6673** (also called AAMA DXF), based on AutoCAD's DXF R13 dialect, with 23 pre-defined layers and pattern pieces encoded as DXF blocks. Each piece outline must be a closed polyline; piece system text and annotation belong on their layers. ASTM D6673 was technically withdrawn in 2019 but remains the dominant interchange format. The Python library ezdxf is the pragmatic way to emit it from our compiler output.

Part 4 • Nesting (the part we replace CLO3D for)

Nesting is the post-grading layout step: once a pattern is graded to a target size, its panels must be arranged on a fabric roll (or a laser bed) so the total fabric consumed is minimized. Industrial markers nest hundreds of panels at once. An experienced human marker-maker on a complex garment achieves 80–85% fabric utilization. Modern automatic nesting in Optitex, Lectra Diamino, Gerber AccuMark, and CLO3D's built-in marker reaches 95–99% utilization — that delta is real money for any atelier that cuts more than a sample a week.

The geometry: No-Fit Polygon (NFP)

The core algorithmic primitive is the **no-fit polygon**. Given two polygons A and B, the NFP of B around A is the locus of positions for B's reference point such that B touches but never overlaps A. Constructed via Minkowski sums, it reduces the question *do these two shapes overlap?* from full geometric intersection testing (slow, error-prone on concave shapes) to a single point-in-polygon test against the NFP (fast, robust). For a nest of n panels you precompute the NFPs once and then place panels by sliding them to the NFP boundary. NFP is the reason modern nesting is tractable; without it, irregular-shape bin packing is a brutal combinatorial problem.

The search: genetic algorithms over NFP placements

NFP gives you a way to place one panel against another without overlap. It does not, on its own, tell you the *best* placement order, rotation, or sheet fit. That is solved by metaheuristics — genetic algorithms (SVGnest, Deepnest), simulated annealing (classic research approach), and ant-colony hybrids. The genetic-algorithm version maintains a population of placement orderings and rotations, evaluates each by simulating the nest end-to-end with bottom-left placement against precomputed NFPs, and evolves the population over generations. SVGnest uses this orbital-NFP-plus-genetic-algorithm approach from Burke et al. 2006; Deepnest extends it with critical sections written in C for speed and adds common-line merging so a laser doesn't cut the same path twice.

Manufacturing constraints the algorithm has to respect

A nest that ignores garment rules is fast but produces unusable cuts. The non-negotiable constraints for apparel nesting:

Grain direction. Every panel has a grainline. The panel can be flipped 180° but never freely rotated, because fabric stretch and drape are direction-dependent. SVGnest exposes a discrete-rotation parameter for this.

Fabric width. The sheet has a fixed width (the bolt) and effectively unbounded length. Panels that exceed the bolt width are constraint violations, not nesting failures.

Kerf and spacing. Laser cutters consume material with each cut. The kerf (typically 0.1–0.3 mm for a CO₂ laser on fabric) plus designer-set spacing becomes the minimum gap between panels. Deepnest's

space between parts setting handles this directly.

Bilateral pairs. Most garment panels come as mirror pairs (left front / right front). The nester must accept paired inputs and respect the pairing in placement.

Fabric features. Plaids, stripes, and one-way prints constrain placement even further. Out of scope for v0.11; revisit when we cut printed fabric.

Commercial vs. open-source landscape

Engine	Licence	Algorithm	Runs in browser	Fit for Maya
Optitex Marker (Nest++)	Commercial	NFP + multi-strategy	No	We are replacing this
Lectra Diamino	Commercial	Proprietary	No	We are replacing this
CLO3D Nest Patterns	Commercial	Proprietary	No	We are replacing this
SVGnest	MIT (open)	NFP + genetic	Yes — pure JS	Phase A baseline
Deepnest (Jack000)	Open source	NFP + GA + C critical sections	Desktop only	Future reference
Deepnest-next	Community fork	Same, maintained	Desktop / electron	Track for v1
NShaker	Commercial	GA + multi-criteria	No	Industrial bench

For Maya's Op Room, **SVGnest** is the right starting point: pure JavaScript, MIT-licensed, browser-only (no Electron, no native binary), and built on the same NFP-plus-genetic-algorithm foundation as Deepnest. Performance will be slower than Deepnest on large layouts, but for a single garment (~10–30 panels) it is comfortably interactive. Deepnest itself stays on the table as the upgrade path if we ever need to nest a production marker (one style, dozens of sizes, hundreds of panels) — at that point we either embed it via Electron in a desktop wrapper or rebuild its C critical sections as WebAssembly.

Part 5 • The competitive landscape (May 2026)

A short read on the products closest to where Maya is going. The relevant axes for our positioning: do they understand bespoke / avant-garde construction, do they emit real cutting files, are they locked to a proprietary library, and what kind of designer do they assume?

Product	Input	Output	Lock-in	Where it leaves room for Maya
CLO3D	Manual 2D drafting	DXF, PDF, plot	Mature, expensive, legacy	Bespoke workflow, voice-driven consultation, a
Style3D AI	Image or sketch	3D mesh + 2D pattern	Closed platform, subscription	Maya's editorial / atelier voice; one-pager + client f
Synthesis (Six Atom)	Existing DXF + size	Graded marker, production-ready	Locked body, modular library	Maya designs from imagination first; Synthesis ass
StitchLift	Photo or text	PDF pattern, XS–XL	Consumer-grade, no atelier	Maya is bespoke, not size-graded mass; ours is on
Optitex / Lectra / Gerber	DXF	DXF + marker + plot	Heavy industrial CAD	Maya is the layer that feeds them; we are upstream

Reading this honestly: Maya's positioning is not *another CLO3D* nor *another StitchLift*. It is **the upstream consultation-to-cut layer** — a couture-grade conversation that ends in a real laser-cuttable file. The pieces the field has already solved (DXF export, NFP-based nesting, parametric pattern languages) should be assembled inside Maya rather than reinvented. Where the field has not solved anything — voice-driven moodboard, three-view atelier rendering, an Operating Room that mirrors the studio's working language — is where Maya keeps its identity.

Part 6 • Implementation plan for the backend Op Room

Phase A • Real nesting on uploaded SVG panels (this week)

Land a working nest without waiting for the upstream geometry pipeline. Wire SVGnest (or a minimal NFP + bottom-left implementation derived from it) into backend.html. Add an upload affordance next to the existing Nest button so a designer can drop a folder of SVG panels — exported from Illustrator, Inkscape, or any 2D CAD — and watch them nest live on the Pattern Window's second page. Expose the manufacturing knobs explicitly: **sheet width** (default 150 cm, common bolt), **spacing** (default 5 mm, covers laser kerf plus seam allowance margin), **allowed rotations** (default 0° / 180° for grain), and **iteration budget** (genetic algorithm generations — exposed so designers can trade runtime against utilization). Download the nested layout as a single laser-ready SVG.

Definition of done. A designer can: pick a client folder; navigate to the active garment piece; press *Upload panels* on the Nest page; drop one or more SVG files; watch the nest run; tweak sheet width and spacing; see the fabric-utilization percentage update; download the result. No regression on the existing pixel-pattern Dissect path. Backend version meta bumps to v0.11.

Phase B • Structured pattern output (next 3–5 weeks)

Replace the gpt-image-2 Dissect call with a gpt-4.1 call that returns GarmentCode-style JSON for the active piece. Compile that JSON into SVG panels via PyGarment, exposed as either a Vercel Python serverless function or, preferably for offline operation, Pyodide running PyGarment in-browser. Auto-populate the Nest page with the compiled panels so the designer rarely uploads by hand. Add a third Pattern Window page — *JSON* — that shows the structured output and lets the designer edit it directly when the LLM misreads a detail.

Definition of done. A designer presses Dissect and sees three pages: Pattern (the existing pixel preview, demoted to a thumbnail-sized reference), JSON (the structured GarmentCode configuration, editable), and Nest (live auto-nest of the compiled panels). Export emits DXF ASTM D6673 via ezdx for industry interchange and laser-ready SVG for direct cutting. Backend version meta bumps to v0.13 (skipping v0.12 to mark the architectural shift).

Phase C · Local foundation-model parser (later)

Once Maya has GPU infrastructure — either a dedicated machine in the atelier or a GPU-bearing Vercel deployment — swap the GPT-4.1 JSON call for a self-hosted ChatGarment or AIpparel checkpoint. The structural shape stays the same: parser outputs JSON, compiler turns it into geometry. Only the parser changes. This is the point at which Maya stops paying per-call to OpenAI for the structural work and gains real-image robustness on photographs of completed garments (closing the loop on the long-term paired-training-data moat).

Phase D · Manufacturing post-processor (parallel to B and C)

The academic research does not supervise grainline, seam allowance, notches, ease policy, or closure templates. We have to. Build a deterministic rule engine that consumes the compiled SVG panels and injects: grainlines (centered, oriented per panel grain), seam-allowance offsets (configurable per seam type — flat-fell, French, plain — and per fabric), notches at every seam pair, closure templates (zipper plackets, button extensions, elastic waist channels), and an ease check that compares finished-garment measurements to the client's body block before the nest is unlocked. Each rule is unit-tested. This module is the part of Maya that *knows* the atelier, and it is where Mana Siyo's house style is encoded.

Phase E · Validation harness

Five validations the field does not standardize but a production system needs, lifted from the prior dossier and adapted to Maya:

Check	Pass target
Panel loop validity (closed polygon, consistent winding)	100%
No self-intersections within a panel	100%
Seam bijection — every stitch pair cross-panel and length-compatible	100%

Check	Pass target
Seam-length mismatch before allowance	median under 1% or 3 mm
Simulation success rate (Warp or similar)	over 95%
Grainline present on every panel	100%
Seam allowance present where required	100%
Closure metadata present when needed	100%
Nest fabric-utilization (per category)	tops > 85%, pants > 88%, simple > 90%

Part 7 · What gets built today, and what does not

Built today

Phase A in full. Backend version meta bumps to v0.11. The Pattern Window's second page becomes a live nesting surface, not a placeholder. The designer can drop panel SVGs, configure sheet width and spacing, run a real NFP-plus-genetic-algorithm nest, see utilization, and download the result as a single laser-ready SVG. The aesthetic stays inside the locked design vocabulary — capsule pills, liquid-glass, centered text, Jost 10px / 0.15em UPPERCASE, Cormorant for editorial paragraphs.

Deferred deliberately

GarmentCode JSON output, PyGarment integration, ChatGarment / AIpparel local inference, DXF ASTM export, the manufacturing post-processor, and the validation harness all wait for the next sessions. The shape of the Op Room (Item Window + Pattern / Nest pages + Dissect / Nest controls) is already correctly named for what comes next, so nothing built today has to be undone.

Why the staging matters

A working nest is the part the atelier feels first. If a designer can drop SVG panels and watch them lay out, that is the moment the Op Room stops feeling like a moodboard and starts feeling like a workroom. Everything upstream of that — real GarmentCode geometry, local foundation-model parsing, grainline injection — is invisible until the nest is real. Phase A makes the rest of the work visible.

Appendix · Reading list and sources

Primary research papers and repositories

- GarmentCode: Programming Parametric Sewing Patterns — Korosteleva et al., SIGGRAPH Asia 2023. ACM TOG, doi:10.1145/3618351. The canonical DSL. GitHub: [maria-korosteleva/GarmentCode](https://github.com/maria-korosteleva/GarmentCode) (PyPI: `pygarment`).
- GarmentCodeData — Korosteleva, Sorkine-Hornung. arXiv:2405.17609. The training substrate. Made-to-measure garments at scale.
- Design2GarmentCode (CVPR 2025) — Style3D / Zhejiang University. arXiv:2412.08603. Program-synthesis from multimodal prompts. github.com/Style3D/design2garmentcode-impl.
- ChatGarment (CVPR 2025) — Bian, Feng, Taheri, Black. arXiv:2412.17811. Real-image VLM parser → GarmentCode JSON. github.com/biansy000/ChatGarment.
- Aipparel — arXiv:2412.03937, MIT-licensed weights. Multimodal foundation model on 120k+ garments. github.com/Sayvai-io/Aipparel.
- NGL-Prompter — arXiv:2602.20700. Training-free structured prompting; design template, not yet deployable.
- GarmentDiffusion — arXiv:2504.21476. Fast vectorized sewing-pattern generation, ~1 s on A10.
- SewFormer / SewFactory — arXiv:2311.04218. The strongest open image-to-panel baseline; useful as teacher.
- Panelformer (WACV 2024) — MIT-licensed panel/stitch predictor.
- NeuralTailor — arXiv:2201.13063. 3D-cloud-to-pattern; foundational evaluation language.

Nesting and manufacturing

- SVGnest — Jack000, MIT. Pure-JS NFP + genetic algorithm. svgnest.com, github.com/Jack000/SVGnest.
- Deepnest — Jack000. Desktop nester with C critical sections; common-line merging. deepnest.io, github.com/Jack000/Deepnest. Community fork: [deepnest-next/deepnest](https://github.com/deepnest-next/deepnest).
- Burke E.K. et al., 2006 — orbital NFP + genetic algorithm; the SVGnest reference. Journal of the Operational Research Society.
- Applying Simulated Annealing and the No Fit Polygon to the Nesting Problem (ResearchGate, 2953145).
- Algorithms for 2D nesting problem based on the no-fit polygon (IEEE Xplore, 6859732).
- ASTM D6673-10 — Standard Practice for Sewn Products Pattern Data Interchange. Withdrawn 2019 but still the dominant interchange.
- ezdxf — Python library for writing AAMA / ASTM-compliant DXF. github.com/mozman/ezdxf.

Commercial landscape (May 2026)

- Optitex Marker / Nest++ — optitex.com/products/marker.

- Lectra Diamino Fashion — marker-making and grading.
- CLO3D Nest Patterns — support.clo3d.com/hc/en-us/articles/115000565568.
- Style3D AI — style3d.ai/blog. AI pattern-making landscape report 2026.
- Synthesis by Six Atomic — sixatomic.com/synthesis. DXF-in / graded-marker-out.
- StitchLift — stitchlift.com. Consumer-grade image-to-PDF pattern.
- NShaker — n-hega.com. Commercial NFP + GA + multi-criteria nester.

Companion document: *Image to Garment Pattern Generation for a Local Backend System* (Research/, May 2026) — the upstream half. This dossier picks up at **Part 2** of that one and reframes the rest of the pipeline around what Maya's backend has to ship.